🧮 Section 5 — Resource Limits with cgroups

Control groups (cgroups) are the second major kernel mechanism—alongside namespaces—that enables containers.

While namespaces provide *isolation of views*, cgroups provide *isolation of resources*.

This section introduces the concepts, structure, and examples.

5.1 What Are cgroups?

Control Groups (cgroups):

- A Linux kernel feature for **accounting**, **limiting**, and **prioritizing** resource usage.
- Applies to *groups of processes* rather than individual processes.
- Used by container runtimes to enforce:
    - CPU limits
    - memory quotas
    - I/O bandwidth limits
    - process count caps
    - device access rules

Key idea:

> Namespaces isolate *what a process can see*

5.2 cgroups v1 vs cgroups v2 (Simplified)

cgroups v1

- Multiple controllers mounted separately.
- Hierarchical but fragmented.
- Common in older systems and still supported.

cgroups v2

- Unified hierarchy (single mount point).
- Simplified model and improved delegation.
- Default on many modern distributions.

5.3 What Can cgroups Limit?

Resource	Description
CPU	Throttle CPU time or assign shares/weights
Memory	Hard memory caps, soft limits, OOM handling
pids	Maximum number of processes inside a group
blkio	Control block device I/O rates
devices	Allow/deny device access
hugetlb	Manage huge page allocations
freezer	Suspend/resume groups of processes
net_cls / net_prio	Tag or prioritize network traffic

Container runtimes automate configuration of these controllers

5.4 Simple Docker-Based Demonstration

Example: CPU throttling

Run a CPU-intensive workload:

```
docker run --rm -it --cpus="0.5" ubuntu bash
```

Inside the container:

```
yes > /dev/null
```

Behavior:

- The process is limited to approximately **50% of a single CPU core**.
- The host scheduler enforces this through cgroups.

Example: Memory limits

```
docker run --rm -it --memory="256m" ubuntu bash
```

Inside the container:

```
# Allocate memory until OOM
python3 -c "a=['x'*10**6]*500"
```

Behavior:

- When the container exceeds ~256 MB, the kernel's OOM killer terminates the process.
- This happens *inside the cgroup*, not affecting the host.

5.5 Inspecting cgroups via Docker

For a running container:

```
docker inspect <container-id>
```

Look for:

- `"CpuQuota"`
- `"CpuShares"`
- `"Memory"`
- `"MemorySwap"`
- `"PidsLimit"`

On the host:

```
cat /sys/fs/cgroup/<controller>/<path>/
```

5.6 OS Concepts Illustrated by cgroups

1. Resource Accounting

The kernel tracks:

- CPU time consumed
- memory usage
- I/O rates
- process count

These statistics are visible under `/sys/fs/cgroup`.

2. Resource Limiting

The kernel enforces:

- CPU quotas
- memory caps
- device access
- process limits

This prevents one group of processes from exhausting system resources.

3. Resource Isolation

Even without namespaces, cgroups can isolate **resource consumption**, ensuring:

- fairness
- QoS
- protection against runaway processes

This is especially relevant for multi-tenant systems.

4. Hierarchy and Propagation

cgroups form a hierarchy:

- limits propagate downward
- child cgroups cannot exceed parent limits

This allows structured control of grouped workloads.

5.7 Minimal Example Without Docker (Optional)

Using `systemd-run` on modern Linux:

```
systemd-run --scope -p MemoryMax=200M -p CPUQuota=30% bash
```

Inside this new shell:

```
stress --cpu 4 --vm 1 --vm-bytes 500M
```

Result:

- CPU threads are throttled
- Memory allocation hits a hard cap

Shows cgroups behavior independent of container tooling.

5.8 Summary

- cgroups regulate **how much** a group of processes can use.
- Together with namespaces, they form the foundation of container runtime isolation.
- Containers provide:
  - CPU control
  - memory limits
  - process-count limits
  - I/O shaping
  - controlled device access
- These mechanisms demonstrate resource management at the kernel level.